

Service-to-Service Communication with Feign + Eureka + Automatic Load Balancing

After exploring **DiscoveryClient** and **LoadBalancerClient**, I've now implemented microservice communication using **Feign + Eureka**, where:

- No service needs to know **hostnames or ports**
- No manual **URL construction**
- No custom **load balancing logic**
- Communication feels like **calling a normal Java method**, but it's actually a call to another microservice 🎯

This is much closer to how **real-world production microservices** are built.

Recap: What Was Done in Earlier Approaches?

1 Using **DiscoveryClient**

- The consumer service would:
 - Ask Eureka: "Give me all instances of **Student-Service** "
 - Pick one instance manually (e.g., the first one)
 - Build the URL using that instance's host and port
 - Call it using a REST client

Good for learning, but:

- We manually chose the instance
- We manually built the URL

- We wrote boilerplate HTTP calling code
-

2 Using `LoadBalancerClient`

- The consumer now:
 - Asked Eureka indirectly through `LoadBalancerClient`
 - Let the load balancer choose an instance (round-robin)
 - Still constructed the URL manually
 - Still used a REST client to make the call

This added **automatic load balancing**, but:

- We still did URL handling
 - We still wrote low-level HTTP client code
-

3 Now: Using Feign + Eureka + Client-Side Load Balancing

In this stage, we moved to using **OpenFeign**, a declarative HTTP client integrated with Spring Cloud.

Instead of:

- Discovering instances manually
- Picking an instance manually
- Constructing URLs manually
- Calling REST via low-level HTTP code

...we simply **define an interface** that describes “what we want to call,” and Feign takes care of **how** it’s called.

Core Ideas

- We interact with **service names**, not host/port
- Eureka handles **service discovery**

- Ribbon / Spring Cloud load balancer (under the hood) handles **instance selection**
- Feign handles:
 - Creating the HTTP request
 - Calling the correct instance
 - Converting JSON to Java object (or string) and back

As a result, the consumer service behaves like:

"I don't care where Student-Service is running or how many instances it has.
I just know its service name. Get me the data."

Architecture Overview

Components

- **Eureka Server**
 - Central service registry
 - All services register here
 - Knows which instances of each service are UP
- **Student-Service (Producer)**
 - Provides student details (roll no, name, percentage, port, instanceId, etc.)
 - Multiple instances can run (e.g., 9091 , 9092 , 9093)
 - Registers itself with Eureka as Student-Service
- **Result-Service (Consumer)**
 - Asks Student-Service for student details
 - Does **not** know where Student-Service is running
 - Uses Feign + Eureka for communication

Detailed Request Flow (Step-by-Step)

When a client calls the **Result-Service** endpoint (for example: `/result/info`):

1. Client → Result-Service

- The client (Postman, browser, another service, etc.) hits the Result-Service endpoint.
- Result-Service receives the request.

2. Result-Service → Feign Client

- Within Result-Service, instead of building a URL or using `RestTemplate`, it simply calls a method on a **Feign client interface**.
- This feels like a normal method call in Java, but it triggers an HTTP request under the hood.

3. Feign Client → Eureka

- Feign sees that the target is a service with a given **service name** (e.g., `Student-Service`).
- It asks Eureka: "Give me a live instance of `Student-Service`."

4. Eureka → Feign / Load Balancer

- Eureka returns metadata for all alive instances of `Student-Service` (e.g., `9091`, `9092`).
- The client-side load balancer (part of Spring Cloud) chooses **one instance** using round-robin or another strategy.

5. Feign → Student-Service Instance

- Feign constructs the complete URL using the selected instance's host + port + API path.
- It makes the HTTP call to that instance.
- The instance processes the request and returns the response (e.g., student JSON).

6. Student-Service → Result-Service → Client

- The response is returned to Result-Service.

- Result-Service may add its own prefix text or formatting.
- The final response is sent back to the original client.

At no step did we manually:

- Look up service instances
 - Build URLs
 - Handle host/port complexity
-

Proving Load Balancing Works

To verify that **client-side load balancing** is working:

1. Start multiple instances of `Student-Service`, e.g.:
 - Instance 1 → port 9091
 - Instance 2 → port 9092
2. Hit the Result-Service endpoint multiple times:
 - `/result/info`
 - Refresh again and again
3. Observe the response:
 - Sometimes you will see `portNo: 9091`
 - Sometimes `portNo: 9092`
 - It keeps alternating between instances

This clearly proves:

- Eureka is aware of **multiple instances**
- Feign client is using the **load balancer**
- Requests are being distributed **in round-robin fashion**

So the system isn't just "working" — it's **scaling and balancing** like a real distributed system.

Comparison of All Three Approaches

Aspect	DiscoveryClient	LoadBalancerClient	Feign + Eureka
Service discovery	Manual via Eureka	Via load balancer	Automatic
Instance selection	Manual (you choose)	Automatic	Automatic
URL building	Manual string concatenation	Manual string concatenation	No URL handling in user code
HTTP call implementation	Manually using REST client	Manually using REST client	Generated by Feign
Boilerplate code	High	Medium	Very low
Production friendliness	Okay for learning	Better	Best & most practical

What I Learned from This Step

- **Service names > URLs**

Communication should be based on **logical service names**, not physical locations (ports, hosts). That's what makes systems cloud-friendly.

- **Client-side load balancing is powerful**

The caller (Result-Service) automatically picks healthy instances. If one instance goes down, traffic shifts to others.

- **Abstraction is our friend**

Feign hides:

- HTTP method setup
- JSON serialization/deserialization
- URL building
- Request execution

letting me **focus on business logic**, not infrastructure plumbing.

- **This is how real microservices are written**

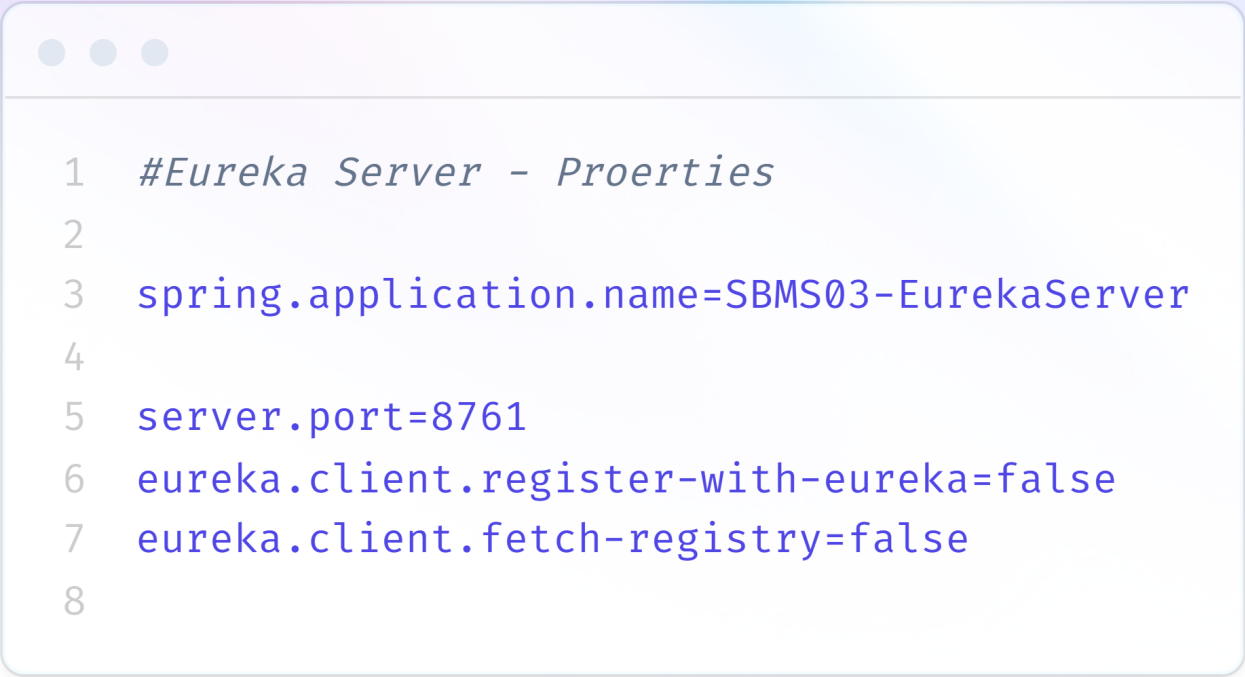
This approach is closer to actual production setups used in companies that heavily rely on Spring Cloud.

Final Thought

Using **Feign + Eureka** shifts microservices from calling fixed URLs to communicating dynamically through service names, allowing automatic load balancing and fault-tolerant scaling. It replaces networking complexity with clean, business-focused code — exactly how real cloud-native systems are designed.

 **Next pages contains the complete practice code for this implementation.**

```
1  //Eureka Server - Main
2  package com.shivraj;
3
4  import org.springframework.boot.SpringApplication;
5  import org.springframework.boot.autoconfigure.SpringBootApplication;
6  import org.springframework.cloud.netflix.eureka.server.EnableEurekaServer;
7
8  @SpringBootApplication
9  @EnableEurekaServer
10 public class Sbms03EurekaServerApplication {
11
12     public static void main(String[] args) {
13         SpringApplication.run(Sbms03EurekaServerApplication.class, args);
14     }
15
16 }
```

```
1  #Eureka Server - Proerties
2
3  spring.application.name=SBMS03-EurekaServer
4
5  server.port=8761
6  eureka.client.register-with-eureka=false
7  eureka.client.fetch-registry=false
8
```

```
1  //Student Service - Producer
2
3  package com.shivraj;
4
5  import org.springframework.boot.SpringApplication;
6  import org.springframework.boot.autoconfigure.SpringBootApplication;
7  import org.springframework.cloud.client.discovery.EnableDiscoveryClient;
8
9  @SpringBootApplication
10 @EnableDiscoveryClient
11 public class Sbms03StudentServiceApplication {
12
13     public static void main(String[] args) {
14         SpringApplication.run(Sbms03StudentServiceApplication.class, args);
15     }
16 }
```

```
1  //Student Service - Producer - Controller
2
3  package com.shivraj.controller;
4
5  import org.springframework.beans.factory.annotation.Value;
6  import org.springframework.http.HttpStatus;
7  import org.springframework.http.ResponseEntity;
8  import org.springframework.web.bind.annotation.GetMapping;
9  import org.springframework.web.bind.annotation.RequestMapping;
10 import org.springframework.web.bind.annotation.RestController;
11
12 import com.shivraj.model.Student;
13
14 @RestController
15 @RequestMapping("/student-api")
16 public class StudentController {
17
18     @Value("${server.port}")
19     private int port;
20
21     @Value("${eureka.instance.instance-id}")
22     private String instanceId;
23
24     @GetMapping("/studentInfo")
25     public ResponseEntity<Student> getStudentDetail() {
26         Student student = new Student(101, "Shivraj Gawande", 95.0, port,
instanceId);
27         return new ResponseEntity<>(student, HttpStatus.OK);
28     }
29 }
```



```
1  #Student Service - Producer - Properties
2
3  server.port=9092
4  spring.application.name=Student-Service
5  eureka.client.service-url.default-zone=http://localhost:8761/eureka
6  eureka.instance.instance-id=${spring.application.name}:${random.value}
7
```

```
1  //Result Service - Consumer
2
3  package com.shivraj;
4
5  import org.springframework.boot.SpringApplication;
6  import org.springframework.boot.autoconfigure.SpringBootApplication;
7  import org.springframework.cloud.client.discovery.EnableDiscoveryClient;
8  import org.springframework.cloud.openfeign.EnableFeignClients;
9
10 @SpringBootApplication
11 @EnableDiscoveryClient
12 @EnableFeignClients
13 public class Sbms03ResultServiceApplication {
14
15     public static void main(String[] args) {
16         SpringApplication.run(Sbms03ResultServiceApplication.class, args);
17     }
18 }
```

```
1  //Result Service - Consumer - Main
2
3  package com.shivraj.client;
4
5  import org.springframework.cloud.openfeign.FeignClient;
6  import org.springframework.web.bind.annotation.GetMapping;
7
8  @FeignClient("Student-Service")
9  public interface StudentInfoConsumerClient {
10
11      @GetMapping("/student-api/studentInfo")
12      public String getStudentInfo();
13  }
```

```
1  //Result Service - Consumer - Controller
2
3  package com.shivraj.controller;
4
5  import org.springframework.beans.factory.annotation.Autowired;
6  import org.springframework.http.HttpStatus;
7  import org.springframework.http.ResponseEntity;
8  import org.springframework.web.bind.annotation.GetMapping;
9  import org.springframework.web.bind.annotation.RequestMapping;
10 import org.springframework.web.bind.annotation.RestController;
11
12 import com.shivraj.client.StudentInfoConsumerClient;
13
14 @RestController
15 @RequestMapping("/result")
16 public class ResultController {
17
18     @Autowired
19     private StudentInfoConsumerClient client;
20
21     @GetMapping("/info")
22     public ResponseEntity<String> getStudentInfo(){
23         String response = "Student Details and result :
24 \n"+client.getStudentInfo();
25
26         return new ResponseEntity<>(response,HttpStatus.OK);
27     }
28 }
```



```
1  #Result Service - Consumer - Properties
```

```
2
```

```
3  server.port=8080
```

```
4  spring.application.name=Result-Service
```

```
5  eureka.client.service-url.default-zone=http://localhost:8761/eureka
```



```
1  // Model Class
2
3  package com.shivraj.model;
4
5  import lombok.AllArgsConstructor;
6  import lombok.Data;
7  import lombok.NoArgsConstructor;
8
9  @Data
10 @AllArgsConstructor
11 @NoArgsConstructor
12 public class Student {
13
14     private int roll;
15     private String name;
16     private double percentage;
17     private int portNo;
18     private String instanceId;
19
20 }
```